ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey

Lab 6 — Your First DataFrame (Final Lab)

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Lab 6 of 6 (Final Lab)
Focus	Building a class roster as a pandas DataFrame, then exploring, filtering, and summarizing it — the same everyday moves used on every real dataset.

Before You Start

This final lab reuses the same class-scores idea from Lab 5, but now as a proper pandas table with student names attached. If you can build a DataFrame, look inside it, filter it, and summarize it with groupby() by the end of this lab, you've got a real, practical pandas toolkit.

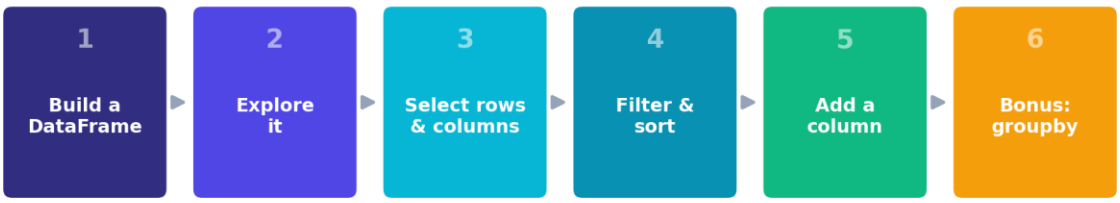

Figure L.1 — Six small steps: build a table, explore it, select from it, filter it, extend it, then summarize it.

✓ QUICK CHECK — Before You Start

Q1. What line of code do you need at the top of every file that uses pandas?

Answer: *import pandas as pd*

Q2. What does pd.DataFrame(a_dictionary) do?

Answer: *Builds a table where each dictionary key becomes a column.*

Q3. What's the difference between .loc and .iloc?

Answer: *.loc selects by row label; .iloc selects by numeric position.*

01 Build Your First DataFrame

A teacher has four students, each with two quiz scores. Build this as a pandas DataFrame from a dictionary and print it.

- *Start your file with import pandas as pd.*
- *Build a dictionary with three keys: "name", "quiz1", "quiz2", each mapping to a list of 4 values.*
- *Pass the dictionary into pd.DataFrame() and store the result in a variable called df.*
- *Print df to see the table.*

AI connection: *This dictionary-to-DataFrame pattern is one of the most common ways real data gets loaded — whether it comes from a database query, an API response, or a spreadsheet.*

Expected output:

```
   name  quiz1  quiz2
0  Amir     90     85
1  Sara     70     60
2   Leo     82     77
3  Nina     60     55
```

02 Explore the Table

Before doing anything else with a new dataset, it's good habit to look at its shape, its column types, and a quick statistical summary. Practice that habit on df from Exercise 01.

- *Print df.shape to see how many rows and columns there are.*
- *Print df.columns to see the column names.*
- *Print df.dtypes to see what type of data each column holds.*
- *Print df.describe() to get quick statistics on the numeric columns.*

AI connection: *This exact sequence — shape, dtypes, describe() — is usually the very first thing done after loading any new dataset, before a single row of analysis happens.*

03 Select Rows and Columns

Practice pulling specific pieces out of the DataFrame — a single column, a couple of columns, and a specific row using both .loc and .iloc.

- *Print df["quiz1"] to get just that column.*
- *Print df[["name", "quiz1"]] to get two columns together.*
- *Create a version of the table indexed by name: df2 = df.set_index("name").*
- *Print df2.loc["Leo"] and df2.iloc[2] — confirm they return the same row.*

AI connection: *Pulling out a handful of columns or a specific row by name is something you'll do constantly while inspecting a dataset or debugging a surprising result.*

04 Filter and Sort

Find the students who scored 80 or above on quiz1, and see the whole class ranked from highest to lowest on quiz1.

- *Build top = df[df["quiz1"] >= 80] and print it.*
- *Print df.sort_values("quiz1", ascending=False) to rank the whole class.*
- *Try combining two conditions: df[(df["quiz1"] >= 70) & (df["quiz2"] >= 70)].*

AI connection: *Filtering for "rows above a threshold" and ranking with sort_values() are two of the most common first steps when exploring any new dataset.*

Expected output:

```
   name  quiz1  quiz2
0  Amir     90     85
2   Leo     82     77
```

05 Add a Column and Handle a Gap

Give every student an average score column and a pass/fail column, then practice handling a missing value.

- *Add df["average"] = (df["quiz1"] + df["quiz2"]) / 2.*
- *Add df["passed"] = df["average"] >= 70.*
- *Print df[["name", "average", "passed"]].*
- *Now set one value to missing: df.loc[1, "quiz2"] = None, then print df.isna().sum() to see how many are missing per column.*
- *Try both filled = df.fillna(0) and dropped = df.dropna(), and print each to see the difference.*

AI connection: *Deciding how to handle a missing value — fill it with something reasonable, or drop the row entirely — is a real decision made at the start of nearly every data project.*

☆ Bonus Stretch Goal — Summarize by Group

The teacher splits the class into two study groups. Use groupby() to compare each group's performance, and save the final table to a CSV file so it can be opened later, even outside Python.

- *Add a new column: df["study_group"] = ["A", "B", "A", "B"] (matching the 4 students in order).*
- *Print df.groupby("study_group")["quiz1"].mean() to compare average quiz1 by group.*
- *Print df.groupby("study_group")["quiz1"].agg(["mean", "max", "count"]) for a fuller summary.*
- *Save the whole table with df.to_csv("class_results.csv", index=False), then read it back with pd.read_csv("class_results.csv") to confirm it round-trips correctly.*

AI connection: *Comparing performance across groups with groupby(), then saving the result with to_csv(), is exactly how a real analysis gets shared with a teammate or handed off to the next step of a pipeline.*

That's the whole track! From print() statements on Day 1 to pandas DataFrames on Day 6 — nice work.